

Smart Pointers & Memory Interview Questions

Բոլոր հնարավոր հարցերը (Memory leak, RAII, new/delete, unique/shared/weak_ptr)

Այս ֆայլը memory management-ի interview հարցերի ամբողջական ժողովածու է: Երեք մաս.

Մաս A — 35 multiple-choice հարց: Մաս B — 30 բաց (open) հարց: Մաս C — 15 tricky կոդ / complexity հարց:

Բոլոր պատասխանները **միայն վերջի էջերում** են (Answer Key A, B, C):

ՄԱՍ A — Multiple Choice (35)

Q1

Memory leak ի՞նչ է: - (a) շատ memory օգտագործել - (b) heap memory վերցված բայց չի ազատվել ու հասցեն կորել - (c) stack overflow - (d) null pointer

Q2

Որ մեկն է memory leak-ի պատճառ: - (a) delete-ի մոռացում - (b) early return առանց delete - (c) pointer overwrite - (d) բոլորը

Q3

Memory leak հայտնաբերող գործիք: - (a) valgrind - (b) AddressSanitizer - (c) երկուսն էլ - (d) ոչ մեկը

Q4

RAII ի՞նչ է նշանակում: - (a) Random Access... - (b) Resource Acquisition Is Initialization - (c) Rapid Allocation... - (d) Reference And Index Init

Q5

RAII-ում resource-ը երբ է ազատվում: - (a) constructor-ում - (b) destructor-ում (scope-ի վերջ) - (c) main-ի վերջում - (d) ձեռքով

Q6

RAII-ի հիմնական առավելությունը: - (a) արագ - (b) exception-safe cleanup, չկա leak - (c) քիչ կոդ - (d) sorted

Q7

new vs malloc հիմնական տարբերությունը: - (a) ոչ մի - (b) new-ը ctor/dtor կանչում, malloc-ը՝ ոչ - (c) malloc-ն ավելի արագ - (d) new-ը C-ի

Q8

new անհաջողության դեպքում ինչ է անում: - (a) return nullptr - (b) throw std::bad_alloc - (c) crash - (d) 0

Q9

malloc անհաջողության դեպքում: - (a) throw - (b) return nullptr - (c) crash - (d) -1

Q10

Կարելի է new-ի հետ free օգտագործել: - (a) այո - (b) ոչ (undefined behavior) - (c) միայն int-ի համար - (d) միշտ

Q11

new[]-ի գույգը ինչ է: - (a) delete - (b) delete[] - (c) free - (d) free[]

Q12

unique_ptr-ի ownership model: - (a) shared - (b) exclusive (մեկ owner) - (c) non-owning - (d) weak

Q13

unique_ptr copy լինում է: - (a) այո - (b) ոչ (միայն move) - (c) միայն int-ի - (d) միշտ

Q14

unique_ptr ownership ինչպես ես փոխանցում: - (a) copy - (b) std::move - (c) get() - (d) reset()

Q15

Նախընտրելի unique_ptr ստեղծում: - (a) unique_ptr(new int) - (b) make_unique() - (c) malloc - (d) new

Q16

shared_ptr-ի ownership model: - (a) exclusive - (b) shared (մի քանի owner) - (c) non-owning - (d) ոչ մի

Q17

shared_ptr-ը երբ է resource-ը delete անում: - (a) առաջին owner-ի ոչնչանալիս - (b) վերջին owner-ի ոչնչանալիս (count 0) - (c) երբեք - (d) main-ի վերջ

Q18

use_count() ինչ է տալիս: - (a) memory չափ - (b) քանի strong owner - (c) քանի weak - (d) bucket

Q19

Նախընտրելի shared_ptr ստեղծում: - (a) shared_ptr(new int) - (b) make_shared() - (c) malloc - (d) new

Q20

make_shared-ի առավելությունը: - (a) 2 allocation - (b) 1 allocation (resource+control block) - (c) ավելի դանդաղ - (d) ոչ մի

Q21

Control block ինչ է պահում: - (a) միայն տվյալ - (b) strong count + weak count + deleter - (c) միայն count - (d) bucket

Q22

shared_ptr-ի ամենամեծ խնդիր: - (a) արագ - (b) circular reference (leak) - (c) copy չկա - (d) null

Q23

Circular reference-ի լուծում: - (a) unique_ptr - (b) weak_ptr - (c) raw pointer - (d) malloc

Q24

weak_ptr ավելացնում է strong count-ը: - (a) այո - (b) ոչ - (c) միայն copy - (d) միշտ

Q25

weak_ptr-ից resource-ի անվտանգ մուտք: - (a) *wp - (b) wp.lock() - (c) wp.get() - (d) wp->

Q26

expired() ինչ է տալիս: - (a) memory չափ - (b) resource ջնջված է? (true/false) - (c) count - (d) bucket

Q27

Default smart pointer ընտրություն: - (a) shared_ptr - (b) unique_ptr - (c) weak_ptr - (d) raw

Q28

Rule of Zero ինչ է: - (a) 0 member - (b) RAII տիպեր -> ոչ մի ձեռքով dtor/copy/move - (c) 0 constructor - (d) 0 allocation

Q29

Rule of Three-ի երեքը: - (a) 3 ctor - (b) dtor, copy-ctor, copy-assign - (c) 3 pointer - (d) 3 member

Q30

Rule of Five-ը ավելացնում է: - (a) 5 ctor - (b) move-ctor + move-assign - (c) 5 member - (d) ոչ մի

Q31

Dangling pointer ինչ է: - (a) null pointer - (b) ջնջված memory-ի ցույց տվող pointer - (c) մեծ pointer - (d) shared

Q32

Double delete ինչ է: - (a) 2 new - (b) նույն pointer 2 անգամ delete (UB) - (c) delete[] - (d) free

Q33

unique_ptr-ի overhead: - (a) մեծ - (b) գրեթե zero (raw-ի չափով) - (c) 2x - (d) atomic

Q34

shared_ptr reference count-ը ի՞նչ է: - (a) սովորական int - (b) atomic (thread-safe) - (c) float - (d) չկա

Q35

release() vs reset() unique_ptr-ում: - (a) նույնը - (b) release ownership հանում (delete չի); reset ազատում (delete) - (c) release delete անում - (d) reset ownership հանում

ՄԱՍ B — Open Questions (30)

1. Memory leak ի՞նչ է, ինչպես է լինում, ինչպես խուսափել:
2. Dangling pointer vs double delete:
3. Memory leak ինչպես էս հայտնաբերում (գործիքներ):
4. RAII ի՞նչ է, ինչպես է աշխատում:
5. RAII ինչպես է աշխատում exception-ի դեպքում (stack unwinding):
6. RAII իրական օրինակներ (memory, file, lock):
7. new vs malloc - ամբողջական տարբերություն:
8. new անհաջողության դեպքում ինչ (bad_alloc):
9. Ինչո՞ւ new-ն ավելի լավ է class-երի համար (constructor):
10. delete vs delete[]:
11. Ինչո՞ւ չի կարելի new+free խառնել:
12. unique_ptr ի՞նչ է, exclusive ownership:
13. Ինչո՞ւ unique_ptr copy չկա:
14. make_unique ինչո՞ւ նախընտրելի:
15. unique_ptr ownership transfer (move):
16. get() vs release() vs reset():
17. shared_ptr ի՞նչ է, shared ownership:
18. Reference counting ինչպես է աշխատում:
19. Control block ի՞նչ է:
20. make_shared vs shared_ptr(new):
21. Circular reference-ի խնդիրը:
22. weak_ptr ի՞նչ է, ինչո՞ւ է պետք:
23. weak_ptr-ով circular reference լուծում:
24. lock() ինչ է անում, ինչո՞ւ պետք:

- 25. expired() ինչ է:
 - 26. Որ smart pointer երբ օգտագործել:
 - 27. Rule of Zero / Three / Five:
 - 28. shared_ptr overhead - ինչու:
 - 29. Smart pointers ինչու են պետք (raw խնդիրներ):
 - 30. Rule of Zero ինչու է լավագույնը:
-

ՄԱՍ C — Tricky Code (15)

C1

```
void f() {  
    int* p = new int(5);  
    return;  
}
```

- (a) OK
- (b) memory leak (delete չկա)
- (c) CE
- (d) crash

C2

```
std::unique_ptr<int> a = std::make_unique<int>(5);  
std::unique_ptr<int> b = a;
```

- (a) OK
- (b) compile error (unique copy չկա)
- (c) runtime error
- (d) leak

C3

```
std::unique_ptr<int> a = std::make_unique<int>(5);  
std::unique_ptr<int> b = std::move(a);  
cout << (a == nullptr);
```

- (a) 0
- (b) 1 (a դարձավ nullptr)
- (c) CE
- (d) UB

C4

```
auto p = std::make_shared<int>(10);  
auto q = p;  
cout << p.use_count();
```

- (a) 1
- (b) 2
- (c) 0
- (d) CE

C5

```
int* p = new int(5);  
delete p;  
delete p;
```

- (a) OK
- (b) undefined behavior (double delete)
- (c) CE
- (d) leak

C6

```
int* p = new int(5);  
free(p);
```

- (a) OK
- (b) undefined behavior (new+free)

- (c) CE
- (d) leak

C7

```
int* arr = new int[10];  
delete arr;
```

- (a) OK
- (b) undefined behavior (delete vs delete[])
- (c) CE
- (d) leak

C8

```
struct Node { std::shared_ptr<Node> next; };  
auto a = std::make_shared<Node>();  
auto b = std::make_shared<Node>();  
a->next = b; b->next = a;
```

- (a) OK, ազատվում
- (b) circular reference -> leak
- (c) CE
- (d) crash

C9

```
auto sp = std::make_shared<int>(10);  
std::weak_ptr<int> wp = sp;  
cout << sp.use_count();
```

- (a) 1 (weak չի ավելացնում)
- (b) 2
- (c) 0
- (d) CE

C10

```
auto sp = std::make_shared<int>(10);
std::weak_ptr<int> wp = sp;
sp.reset();
cout << wp.expired();
```

- (a) 0
- (b) 1 (resource ջնջված)
- (c) CE
- (d) UB

C11

```
std::weak_ptr<int> wp;
{
    auto sp = std::make_shared<int>(5);
    wp = sp;
}
if (auto s = wp.lock()) cout << "alive";
else cout << "gone";
```

- (a) alive
- (b) gone (sp scope-ից դուրս, ջնջված)
- (c) CE
- (d) UB

C12

```
class A { int* p; public: A(){p=new int[10];} ~A(){delete[] p;} };
A a1;
A a2 = a1;
```

- (a) OK
- (b) shallow copy -> double delete (UB) [Rule of Three խախտում]
- (c) CE
- (d) leak

C13

```
A* p = (A*)malloc(sizeof(A)); // A null constructor
```

- (a) constructor կանչվում
- (b) constructor Չի կանչվում (raw memory)
- (c) CE
- (d) crash

C14

```
auto p = std::make_unique<int>(5);  
int* raw = p.release();  
cout << (p == nullptr);
```

- (a) 0
- (b) 1 (release ownership հանեց) + raw պետք է delete
- (c) CE
- (d) UB

C15

```
{  
    auto a = std::make_shared<int>(1);  
    {  
        auto b = a;  
        cout << a.use_count() << " "; // ?  
    }  
    cout << a.use_count(); // ?  
}
```

- (a) 2 1
 - (b) 1 1
 - (c) 2 2
 - (d) 1 2
-

ANSWER KEY A

1-b · 2-d · 3-c · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b 11-b · 12-b · 13-b · 14-b · 15-b · 16-b · 17-b · 18-b · 19-b · 20-b 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b 31-b · 32-b · 33-b · 34-b · 35-b

ANSWER KEY B (Համառոտ)

1. Heap memory վերցված բայց չի ազատվել, հասցեն կորել. Պատճառ՝ delete-ի մոռացում, early return/exception, overwrite. Խուսափել՝ RAII/smart pointers:
2. Dangling՝ ջնջված memory-ի ցույց տվող pointer (use-after-free). Double delete՝ նույն pointer 2 անգամ delete. Երկուսն էլ UB:
3. Valgrind (--leak-check=full), AddressSanitizer/LeakSanitizer (-fsanitize=address/leak), CRT debug heap:
4. Resource Acquisition Is Initialization. resource վերցվում ctor-ում, ազատվում dtor-ում; scope-ի վերջ -> ավտոմատ cleanup:
5. Exception-ի ժամանակ stack unwinding -> C++-ն ավտոմատ կանչում local object-ների destructor-ները -> cleanup. Դրա համար RAII exception-safe է:
6. unique_ptr/shared_ptr (memory), lock_guard (mutex), fstream (file), vector/string:
7. new՝ ctor/dtor կանչում, type-safe, throw bad_alloc, operator. malloc՝ ոչ ctor, void* return, nullptr, function, ունի realloc:
8. throw std::bad_alloc (malloc-ը nullptr):
9. Constructor կանչում -> object ճիշտ init; malloc-ը միայն raw memory (uninitialized):
10. new->delete, new[]->delete[]. Սխալ զույգ -> UB:
11. Տարբեր allocator/mechanism (new operator vs malloc function); խառնելը -> UB:
12. Exclusive ownership -- մեկ resource մեկ owner; scope-ի վերջ ավտոմատ delete:
13. Exclusive ownership -- copy -> երկու owner -> double delete. Միայն move:
14. Ավելի կարճ, exception-safe, չկա բացահայտ new:
15. std::move -> հին pointer nullptr:
16. get() raw (ownership պահում); release() ownership հանում raw տալիս (delete չի); reset() ազատում (delete):
17. Shared ownership -- մի քանի owner; վերջին owner-ի ոչնչանալիս delete (ref count):
18. copy count++, ոչնչանում count--, count 0 delete:

19. Strong count + weak count + deleter; բոլոր copy նույն control block:
20. make_shared 1 allocation (արագ, exception-safe); (new) 2:
21. Երկու shared_ptr իրար ցույց տալիս -> count երբեք 0 -> leak:
22. Non-owning smart pointer, չի ավելացնում count; circular լուծում, observer/cache:
23. weak_ptr մեկ ուղղությամբ (parent->child shared, child->parent weak) -> cycle կոտրվում:
24. Վերադարձնում shared_ptr (եթե կա) կամ nullptr; անվտանգ մուտք:
25. true եթե resource ջնջված (strong count 0):
26. unique default; shared իսկապես shared պետք է; weak circular/observer:
27. Three (dtor/copy-ctor/copy-assign), Five (+move), Zero (RAII -> ոչ մի):
28. Atomic ref count + control block -> unique-ից ծանր:
29. Raw խնդիրների (leak/dangling/double-delete/ownership) լուծում RAII-ով:
30. Ոչ մի ձեռքով special member -> քիչ կոդ, քիչ սխալ, RAII տիպեր:

ANSWER KEY C

C1-b · C2-b · C3-b · C4-b · C5-b · C6-b · C7-b · C8-b · C9-a · C10-b C11-b · C12-b · C13-b · C14-b · C15-a

Բացատրություններ (C)

C1 new առանց delete -> leak: **C2** unique copy չկա -> CE: **C3** move-ից հետո a -> nullptr -> 1: **C4** copy -> use_count 2: **C5** double delete -> UB: **C6** new+free -> UB: **C7** new[] + delete (ոչ delete[]) -> UB: **C8** circular shared_ptr -> leak: **C9** weak չի ավելացնում -> use_count 1: **C10** reset -> resource ջնջված -> expired true (1): **C11** sp scope-ից դուրս -> resource ջնջված -> lock() nullptr -> "gone": **C12** Rule of Three խախտում -> shallow copy -> double delete UB: **C13** malloc -> constructor չի կանչվում: **C14** release -> p nullptr (1), raw-ը պետք է ձեռքով delete: **C15** ներքին scope count 2, դուրս գալուց 1 -> "2 1":

Ընդհանուր կանոններ

LEAK: new առանց delete | RAII/smart pointer -> խուսափել
new->delete | new[]->delete[] | malloc->free | ՄԻ ԽԱՌՆԻՐ (UB)
unique: copy ՉԿԱ (move) | shared: ref count, circular->leak | weak: lock()/expired()
Rule: Zero (RAII) > Five > Three
make_unique / make_shared -> ՆԱԽԸՆՏՐԵԼԻ